ByteGuard: blog de ciberseguridad en Node.js y SQLite para formar hábitos digitales seguros en la Institución Educativa John F. Kennedy

Kristen Apellido y Erick Apellido

Área de Sistemas, Institución Educativa John F. Kennedy, Puerto Boyacá (Boyacá)

Grado Once (A-B-C)

Proyecto de Grado

Profesor: _____

Asesor: _____

__ de noviembre de 2026

Nota del Autor

Este documento presenta ByteGuard, un blog editorial de ciberseguridad que desarrollamos como proyecto de grado del área de Sistemas. El proyecto cubre una aplicación web que funciona en un computador local, donde lectores, escritores y administradores usan el mismo sitio con permisos distintos. Construimos el servidor con Node.js, Express y SQLite, y la interfaz con HTML, CSS y JavaScript. La publicación en un dominio de internet queda fuera de este alcance, aunque el proyecto ya incluye la configuración para hacerlo.

Agradecemos a nuestras familias por su paciencia durante el año, al profesor del área de Sistemas y a nuestro asesor por sus observaciones, y a los compañeros que probaron el blog y nos señalaron errores.

Si quiere comunicarse con nosotros sobre este proyecto, escriba a los siguientes correos electrónicos.

E-mail: correo.kristen@jfkennedy.edu.co y correo.erick@jfkennedy.edu.co

Resumen

ByteGuard es un blog editorial de ciberseguridad en español creado para que estudiantes, docentes y familias de la Institución Educativa John F. Kennedy de Puerto Boyacá aprendan a proteger sus cuentas, sus datos y sus dispositivos. Muchos jóvenes usan redes sociales y correo a diario sin saber reconocer un mensaje de phishing ni activar la verificación en dos pasos. Desarrollamos una aplicación web con Node.js y Express en el servidor, una base de datos SQLite y una interfaz que se adapta a computador y celular. El lector puede buscar artículos por tema, guardarlos y comentarlos. El escritor redacta en un editor con vista previa y envía su texto a revisión, y el administrador lo aprueba o lo devuelve con observaciones antes de publicarlo. Aplicamos controles del OWASP Top 10: contraseñas almacenadas con bcrypt, sesiones revocables, consultas parametrizadas, saneamiento de HTML y límites de solicitudes. Las pruebas automatizadas superaron 72 comprobaciones de la API, además de pruebas de navegador y auditorías de accesibilidad en cinco anchos de pantalla. El blog funciona con seis artículos educativos iniciales y queda listo para que la comunidad escolar lo use y lo amplíe.

Palabras clave

ciberseguridad, blog editorial, Node.js, SQLite, seguridad web, alfabetización digital

Abstract

ByteGuard is a Spanish-language editorial blog about cybersecurity, built so that students, teachers and families at Institución Educativa John F. Kennedy in Puerto Boyacá can learn to protect their accounts, data and devices. Many young people use social media and email every day without knowing how to spot a phishing message or turn on two-step verification. We developed a web application with Node.js and Express on the server, an SQLite database and a responsive interface for desktop and mobile. Readers can search articles by topic, bookmark them and comment on them. Writers draft in an editor with live preview and submit their work for review, and an administrator approves it or returns it with feedback before publication. We applied controls from the OWASP Top 10: passwords stored with bcrypt, revocable sessions, parameterized queries, HTML sanitization and rate limiting. The automated tests passed 72 API checks, plus browser tests and accessibility audits at five screen widths. The blog runs with six initial educational articles and is ready for the school community to use and extend.

Keywords

cybersecurity, editorial blog, Node.js, SQLite, web security, digital literacy

Índice

Introducción 7

Planteamiento del problema 8

Justificación 9

Objetivos 10

Objetivo General 10

Objetivos Específicos 10

Pregunta problematizadora 11

Marco teórico 12

Marco conceptual 12

Marco de antecedentes 13

Marco legal 14

Aspectos Metodológicos 16

Fases de proyecto 16

Cronograma de actividades 17

Técnicas o instrumentos 17

Población 18

Análisis de Resultados 19

Conclusiones 26

Referencias 27

Índice de Tablas

Tabla 1 Cronograma de actividades 17

Tabla 2 Controles de seguridad implementados en ByteGuard 23

Índice de Figuras

Figura 1 Portada de ByteGuard en computador 20

Figura 2 Lectura de un artículo 21

Figura 3 Editor de artículos con vista previa 22

Figura 4 Panel de administración 23

Figura 5 Portada de ByteGuard en celular 25

Introducción

Un estudiante de grado once pasa buena parte del día conectado. Hace tareas en línea, habla por WhatsApp, publica en Instagram o TikTok y crea cuentas en plataformas que le piden correo y contraseña. Cada cuenta guarda fotos, conversaciones y datos personales que un atacante puede aprovechar.

La información sobre seguridad digital existe, pero llega dispersa, a menudo en inglés y con un lenguaje técnico que aleja al lector que más la necesita. En la Institución Educativa John F. Kennedy no teníamos un espacio propio donde estudiantes y docentes pudieran leer y escribir sobre estos temas.

Por eso construimos ByteGuard, un blog editorial en español con artículos cortos sobre phishing, contraseñas, passkeys, copias de seguridad, privacidad e inteligencia artificial. Cualquier persona puede leerlos sin registrarse. Quien quiera aportar crea una cuenta de escritor y envía su artículo a revisión, y un administrador lo revisa antes de publicarlo para evitar información falsa o datos privados expuestos.

El blog también tenía que ser seguro por dentro. Un sitio que enseña ciberseguridad y guarda las contraseñas en texto plano contradice su propio mensaje, así que dedicamos buena parte del trabajo a proteger las cuentas, validar los datos y probar el sistema con herramientas automatizadas.

Planteamiento del problema

Los estudiantes de la Institución Educativa John F. Kennedy usan el celular y el computador para estudiar, comunicarse y entretenerse. Muchos repiten la misma contraseña en varias cuentas, abren enlaces que llegan por mensaje sin revisar el remitente y no saben cómo recuperar una cuenta robada. Estas prácticas los exponen al robo de cuentas, la suplantación de identidad, la extorsión y la pérdida de información.

Colombia castiga estos delitos desde la Ley 1273 de 2009, pero la ley actúa después del daño. La prevención depende de que cada persona reconozca las señales de un fraude y adopte hábitos básicos, como activar la verificación en dos pasos o hacer copias de seguridad.

En la institución, la formación en seguridad digital ocurre en charlas ocasionales o en algunas clases de tecnología. La comunidad no tiene un lugar permanente, en español y con lenguaje cercano, para consultar estos temas cuando los necesita. Los recursos que circulan en redes mezclan información confiable con consejos desactualizados, y casi ninguno deja que los propios estudiantes escriban y compartan lo que aprenden.

Hay además un problema técnico. Un sitio que recibe cuentas y textos de usuarios queda expuesto a ataques comunes como la inyección SQL, el *cross-site scripting* (XSS) o el robo de sesiones. Si los desarrolladores no prevén esos riesgos desde el diseño, el sitio termina poniendo en peligro los datos de las mismas personas a las que quería proteger.

Justificación

Desarrollamos ByteGuard porque la comunidad educativa necesita un recurso propio para aprender a protegerse en internet.

Para los estudiantes, el blog ofrece lecturas cortas y prácticas: cómo reconocer un correo de phishing, por qué conviene usar passkeys o cómo comprobar que una copia de seguridad sí funciona. También les da un espacio para escribir. Explicar un tema a los compañeros obliga a entenderlo primero.

La institución gana una herramienta que los docentes del área de Sistemas pueden usar en clase y que las próximas promociones pueden ampliar. El código está organizado por módulos y viene con un manual de instalación y uso en Word.

Padres de familia y vecinos pueden consultar las mismas guías desde cualquier navegador, en computador o celular, sin crear una cuenta.

El proyecto nos formó a nosotros como autores. Construirlo nos obligó a aplicar lo que el blog enseña: guardar las contraseñas como hash, limitar los intentos de inicio de sesión, validar cada dato que llega del usuario y probar el sistema antes de entregarlo.

Objetivos

Objetivo General

Desarrollar un blog editorial de ciberseguridad en español, construido con Node.js, Express y SQLite, que permita a la comunidad de la Institución Educativa John F. Kennedy leer, escribir y publicar artículos revisados sobre seguridad digital.

Objetivos Específicos

- Investigar los riesgos digitales más frecuentes entre jóvenes y las prácticas para prevenirlos, como base de los contenidos del blog.
- Diseñar una interfaz adaptable a computador y celular, con navegación por teclado y criterios de accesibilidad.
- Programar un servidor con Express y una base de datos SQLite que gestione cuentas, artículos, comentarios y guardados.
- Implementar un flujo editorial con roles de escritor y administrador, en el que cada artículo pase por revisión antes de publicarse.
- Aplicar controles de seguridad basados en el OWASP Top 10 para proteger las cuentas y los datos de los usuarios.
- Verificar el funcionamiento del sistema con pruebas automatizadas de la API, del navegador y de accesibilidad.

Pregunta problematizadora

¿Cómo puede un blog editorial de ciberseguridad, desarrollado con Node.js, Express y SQLite, ayudar a que la comunidad de la Institución Educativa John F. Kennedy de Puerto Boyacá reconozca los riesgos digitales y adopte hábitos seguros en internet?

Marco teórico

Marco conceptual

Ciberseguridad. El NIST (s.f.) la define como la prevención de daños, la protección y la restauración de computadores, sistemas y servicios de comunicación electrónica, incluida la información que contienen, para garantizar su disponibilidad, integridad, autenticación, confidencialidad y no repudio. En ByteGuard la usamos en dos sentidos: como tema de los artículos y como requisito de construcción del sitio.

Phishing. Es un engaño en el que el atacante suplanta a una entidad conocida, como un banco, una red social o la propia institución, para que la víctima entregue sus credenciales o abra un archivo malicioso. Llega por correo, SMS o mensajería, y suele presionar con urgencia (INCIBE, s.f.).

Autenticación, contraseñas y passkeys. La autenticación comprueba que una persona es quien dice ser. La contraseña es el método más común y también el más débil cuando se repite o es corta. Las passkeys reemplazan la contraseña por un par de llaves criptográficas guardadas en el dispositivo, y el sitio nunca recibe un secreto que alguien pueda robar (FIDO Alliance, s.f.).

Aplicación web cliente-servidor. El navegador del usuario (cliente) pide páginas y datos, y un programa en otro equipo (servidor) responde por medio del protocolo HTTP. En ByteGuard, el cliente es la página index.html con su código JavaScript, y el servidor es una aplicación de Node.js.

Node.js y Express. Node.js es un entorno que ejecuta JavaScript fuera del navegador, sobre el motor V8 de Google (OpenJS Foundation, s.f.). Express es un marco de trabajo para Node.js que organiza las rutas y los *middleware*, es decir, las funciones que procesan cada solicitud antes de responderla (Express, s.f.).

API REST. Es una interfaz en la que el cliente accede a recursos mediante rutas y métodos HTTP como GET, POST, PATCH y DELETE. ByteGuard expone rutas como /api/posts para los artículos, /api/auth para las cuentas y /api/admin para la administración.

SQLite. Es un motor de base de datos SQL que no necesita un servidor aparte y guarda toda la base en un solo archivo (SQLite, s.f.). Lo elegimos porque se instala con el proyecto y no exige programas como XAMPP o MySQL.

Hash de contraseñas y bcrypt. Un hash transforma la contraseña en una cadena de la que no se puede recuperar el texto original. Bcrypt es una función de hash diseñada para ser lenta a propósito, lo que encarece los ataques de fuerza bruta (Provos y Mazières, 1999).

JSON Web Token (JWT). Es un formato estándar para transmitir datos firmados entre dos partes (Jones et al., 2015). ByteGuard guarda el token de sesión en una *cookie* HttpOnly y además registra cada sesión en la base de datos, para poder cerrarla cuando el usuario cambia su contraseña o un administrador lo suspende.

OWASP Top 10. Es la lista de los diez riesgos más críticos en aplicaciones web que publica la fundación OWASP (2021). Incluye el control de acceso roto, las fallas criptográficas, la inyección y la configuración insegura, entre otros. Nos sirvió como lista de verificación durante el desarrollo.

Inyección SQL y XSS. En la inyección SQL, el atacante escribe código SQL en un campo de texto para leer o borrar datos. En el XSS, inserta código JavaScript en una página para que se ejecute en el navegador de otros usuarios. Las consultas parametrizadas y el saneamiento de HTML previenen ambos ataques (OWASP Foundation, 2021).

Markdown. Es un lenguaje de marcado ligero que convierte texto plano con símbolos sencillos, como el numeral para un subtítulo o dos asteriscos para la negrita, en HTML (Gruber, 2004). Los escritores de ByteGuard redactan en Markdown y ven el resultado en la vista previa.

Accesibilidad y diseño adaptable. La accesibilidad web busca que personas con discapacidad puedan usar un sitio, y las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2 establecen los criterios para lograrlo (W3C, 2023). El diseño adaptable ajusta la página al tamaño de la pantalla, desde un celular de 360 píxeles hasta un monitor de escritorio.

Marco de antecedentes

WeLiveSecurity (ESET). Es el portal de noticias y guías de seguridad informática de la empresa ESET, con una edición en español para Latinoamérica (ESET, s.f.). Publica artículos sobre fraudes, malware y privacidad escritos por su equipo de investigadores. Tomamos de este portal la organización por categorías y el tono divulgativo. A diferencia de ByteGuard, los lectores no pueden proponer artículos.

Oficina de Seguridad del Internauta (OSI) de INCIBE. El Instituto Nacional de Ciberseguridad de España mantiene este sitio con guías para ciudadanos, avisos sobre fraudes vigentes y materiales para menores y educadores (INCIBE, s.f.). Su enfoque práctico, con pasos concretos para cada riesgo, inspiró la lista de hábitos de seguridad de ByteGuard.

En TIC Confío+ (MinTIC). El Ministerio de Tecnologías de la Información y las Comunicaciones de Colombia promueve con esta estrategia el uso responsable de internet entre niños, jóvenes y familias, con charlas y contenidos sobre ciberacoso, sexting y privacidad (MinTIC, s.f.). Comparte nuestra población objetivo, aunque funciona como programa nacional y no como un espacio de escritura para una comunidad escolar.

Roles editoriales de WordPress. WordPress, el gestor de contenidos más usado en la web, define roles como colaborador, autor, editor y administrador. El colaborador escribe artículos pero no puede publicarlos sin la aprobación de un editor (WordPress.org, s.f.). Adaptamos esa idea a dos roles: el escritor envía su artículo a revisión y el administrador decide si se publica.

Marco legal

Ley 23 de 1982 sobre derechos de autor. Protege las obras literarias, científicas y artísticas, y reconoce al autor derechos morales (ser reconocido como creador) y patrimoniales (decidir sobre la reproducción y distribución de la obra). El Decreto 1360 de 1989 extiende esta protección a los programas de computador, y la Ley 1915 de 2018 actualizó la norma (Congreso de Colombia, 1982, 2018). En ByteGuard la aplicamos de tres maneras: cada artículo educativo cita sus fuentes originales, el blog muestra el nombre del autor de cada publicación y las librerías de terceros que usamos (Express,

better-sqlite3, bcrypt, Helmet, markdown-it, entre otras) tienen licencias libres, como MIT o Apache 2.0, que permiten usarlas siempre que se conserve el aviso de autoría.

Ley 1273 de 2009 de delitos informáticos. Creó en el Código Penal el bien jurídico "de la protección de la información y de los datos" y tipificó conductas como el acceso abusivo a un sistema informático, la interceptación de datos, el daño informático, el uso de software malicioso, la violación de datos personales y la suplantación de sitios web para capturar datos personales (Congreso de Colombia, 2009). Varios artículos del blog explican estos delitos para que el lector sepa reconocerlos y denunciarlos.

Ley Estatutaria 1581 de 2012 de protección de datos personales. Regula el tratamiento de los datos personales y exige autorización del titular, finalidad definida y medidas de seguridad (Congreso de Colombia, 2012). ByteGuard solo pide nombre, correo y contraseña para crear una cuenta, usa esos datos para la cuenta y nada más, y explica su manejo en la sección de privacidad del sitio.

Ley 1620 de 2013 de convivencia escolar. Crea el Sistema Nacional de Convivencia Escolar e incluye el ciberacoso escolar entre las situaciones que las instituciones deben prevenir y atender (Congreso de Colombia, 2013). Los contenidos de ByteGuard sobre privacidad y redes sociales apoyan esa labor preventiva.

Aspectos Metodológicos

Fases de proyecto

Organizamos el trabajo en siete fases durante el año escolar 2026. La fase 06 corre en paralelo con las demás.

- **Fase 01. Investigación y selección del tema (febrero y marzo, 6 semanas):** revisamos fuentes como OWASP, INCIBE y ESET, conversamos con compañeros sobre los riesgos que enfrentan en línea y definimos las categorías del blog: Privacidad, Seguridad, Amenazas, Herramientas, Guía e IA y sociedad.
- **Fase 02. Análisis y diseño (marzo y abril, 6 semanas):** definimos los tres roles (lector, escritor y administrador), el flujo de revisión editorial y el modelo de datos con las tablas de usuarios, artículos, sesiones, guardados, comentarios, historial editorial y suscripciones. Dibujamos bocetos de las pantallas principales.
- **Fase 03. Desarrollo de la interfaz (abril y mayo, 6 semanas):** programamos la portada, el catálogo, la lectura de artículos y los paneles en HTML, CSS y JavaScript. Seguimos la documentación de MDN Web Docs para el diseño adaptable y la navegación por teclado.
- **Fase 04. Servidor y base de datos (mayo a julio, 8 semanas):** construimos el servidor con Express, las rutas de la API y la base de datos SQLite con migraciones que conservan los datos al actualizar. Seguimos las guías oficiales de Express y better-sqlite3.
- **Fase 05. Flujo editorial, seguridad y contenidos (julio y agosto, 7 semanas):** implementamos el editor con vista previa, la revisión y devolución de artículos, el historial editorial, la moderación de comentarios y los controles de seguridad. Escribimos los seis artículos educativos iniciales.
- **Fase 06. Análisis de resultados (febrero a noviembre):** después de cada fase revisamos qué funcionaba, qué fallaba y qué faltaba, y ajustamos el plan.
- **Fase 07. Pruebas, documentación y sustentación (septiembre a noviembre, 10 semanas):** escribimos pruebas automatizadas de la API y del navegador, corregimos los errores

encontrados, redactamos el manual de usuario y este documento, y preparamos la sustentación.

Cronograma de actividades

La Tabla 1 muestra la programación de las fases por mes.

Tabla 1 Cronograma de actividades

FASE	Ene.	Feb.	Mar.	Abr.	May.	Jun.	Jul.	Ago.	Sep.	Oct.	Nov.	Dic.
Fase 01												
Fase 02												
Fase 03												
Fase 04												
Fase 05												
Fase 06												
Fase 07												

La fase 06, de análisis de resultados, se desarrolla en paralelo con las demás fases durante todo el año. En enero y diciembre no programamos actividades porque corresponden a vacaciones escolares.

Técnicas o instrumentos

Para desarrollar y probar ByteGuard usamos los siguientes recursos:

- **Equipos:** computador con Windows 11, conexión a internet y un celular para revisar la versión móvil.
- **Entorno de ejecución:** Node.js 20 y su gestor de paquetes npm.
- **Servidor:** Express 4 para las rutas, better-sqlite3 para la base de datos SQLite, bcrypt para las contraseñas, jsonwebtoken y cookie-parser para las sesiones, Helmet y express-rate-limit para las cabeceras de seguridad y el límite de solicitudes, xss para el saneamiento de HTML y dotenv para la configuración.

- **Interfaz:** HTML5, CSS3 y JavaScript sin marcos de trabajo adicionales, con markdown-it para convertir los artículos a HTML.
- **Pruebas:** Playwright para automatizar Chrome y Edge, y axe-core para las auditorías de accesibilidad.
- **Navegadores:** Google Chrome y Microsoft Edge.
- **Documentación:** Microsoft Word para el manual de usuario y este documento.

Población

El proyecto se dirige a la comunidad de la Institución Educativa John F. Kennedy de Puerto Boyacá (Boyacá). Los lectores principales son los estudiantes de básica secundaria y media, que usan redes sociales y servicios en línea a diario. Los docentes pueden usar los artículos como material de clase, y los padres de familia pueden consultarlos para acompañar a sus hijos.

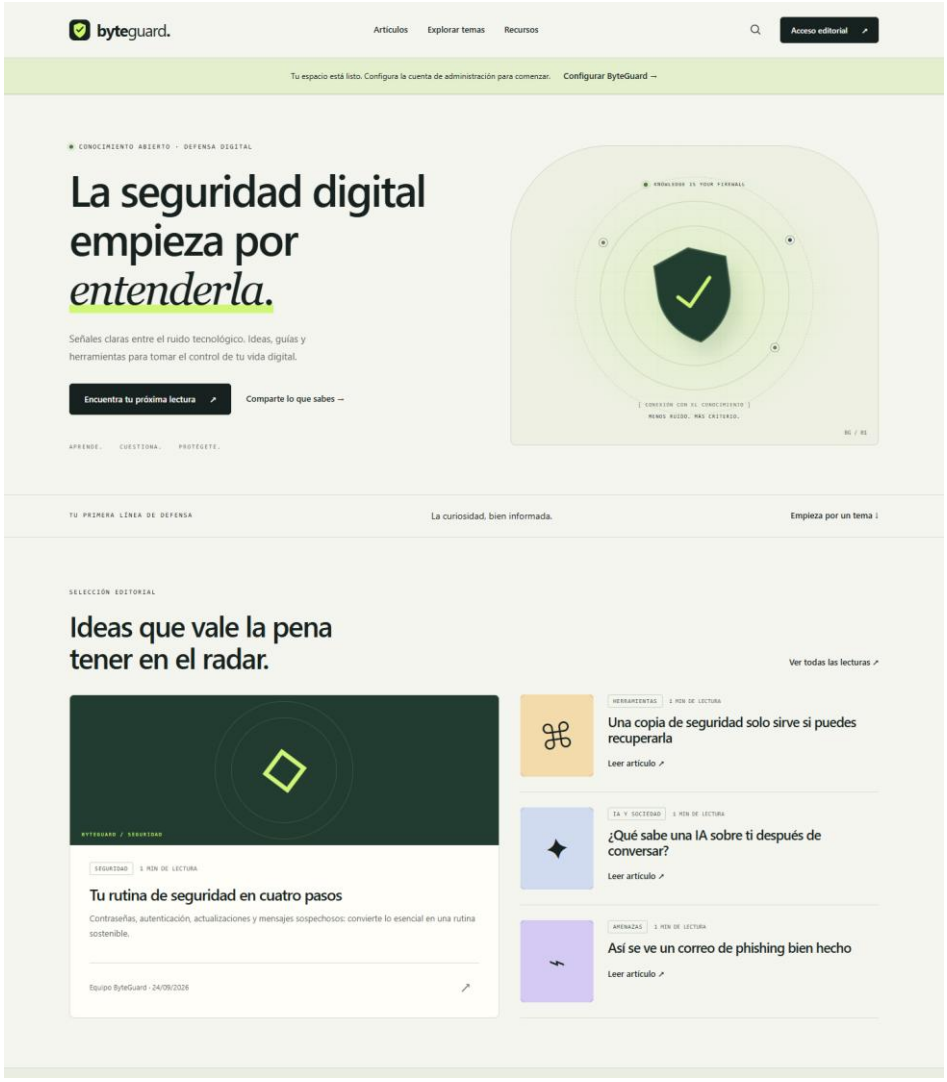
Los escritores son estudiantes y docentes interesados en compartir lo que saben sobre seguridad digital. Los administradores son docentes o estudiantes designados por el área de Sistemas para revisar los artículos y moderar los comentarios.

Análisis de Resultados

Al cierre del proyecto, ByteGuard funciona como un blog editorial completo en un computador local. Esta sección describe el producto obtenido y los resultados de las pruebas.

Producto obtenido. La portada muestra un artículo destacado, el catálogo completo con búsqueda que ignora tildes, filtros por categoría y paginación (ver Figura 1). Cada artículo tiene una dirección propia, índice, autor, fecha, tiempo estimado de lectura y artículos relacionados (ver Figura 2). El lector con cuenta puede guardar artículos y comentar; los comentarios quedan privados hasta que un administrador los aprueba. El blog incluye además una lista de hábitos de seguridad, un canal RSS en /feed.xml y un mapa del sitio para buscadores.

Figura 1 Portada de ByteGuard en computador



Nota. Captura de la parte superior de la portada, generada por la prueba automatizada de navegador del proyecto.

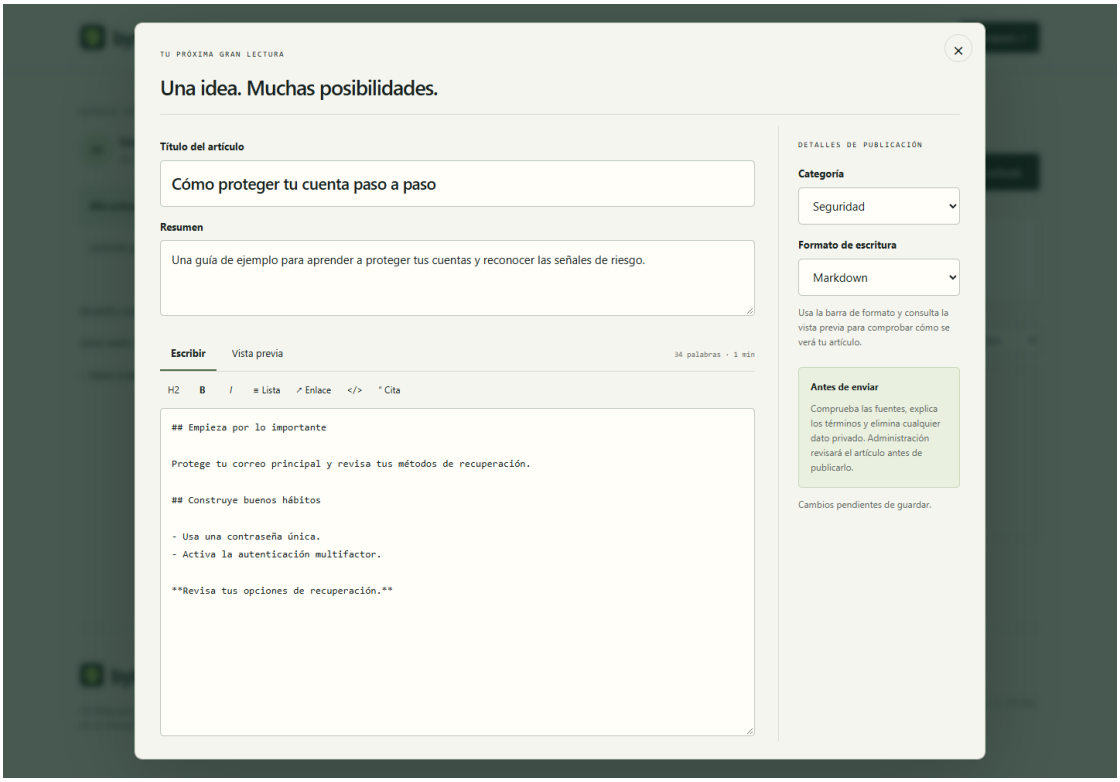
Figura 2 Lectura de un artículo



Nota. Captura de la parte superior de la página del artículo, generada por la prueba automatizada de navegador del proyecto.

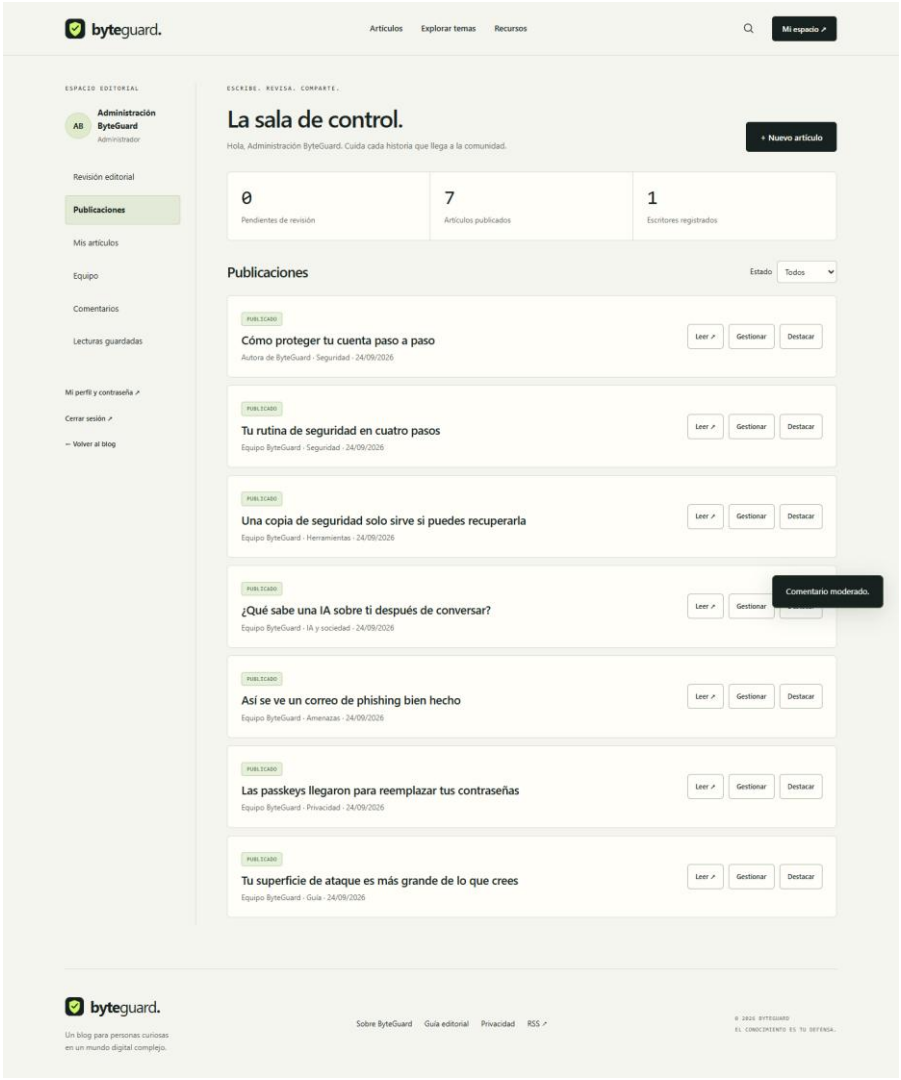
Flujo editorial. El escritor redacta en un editor con barra de formato, contador de palabras y vista previa (ver Figura 3). Guarda borradores y los envía a revisión. El administrador abre el artículo completo desde el panel de administración (ver Figura 4) y lo aprueba o lo devuelve con observaciones. Cada paso queda en un historial editorial, y un control de versión impide que dos personas sobrescriban el mismo artículo sin darse cuenta.

Figura 3 Editor de artículos con vista previa



Nota. Captura generada por la prueba automatizada de navegador del proyecto.

Figura 4 Panel de administración



Nota. Captura generada por la prueba automatizada de navegador del proyecto.

Controles de seguridad. La Tabla 2 relaciona cada control implementado con el riesgo del OWASP Top 10 que mitiga.

Tabla 2 Controles de seguridad implementados en ByteGuard

Control	Riesgo que mitiga	Implementación
Contraseñas con hash	Fallas criptográficas	bcrypt; la base de datos nunca guarda la contraseña en texto plano
Sesiones revocables	Fallas de autenticación	JWT en cookie HttpOnly y SameSite, registrado en la base de datos; el cambio de contraseña y la suspensión cierran las sesiones anteriores
Permisos en el servidor	Control de acceso roto	Cada ruta comprueba el rol de la cuenta; el registro público nunca concede administración

Consultas parametrizadas	Inyección	better-sqlite3 con parámetros en todas las consultas
Saneamiento de HTML	Inyección (XSS)	Librería xss sobre el contenido de artículos y comentarios
Límite de solicitudes	Fallas de autenticación	express-rate-limit en registro, inicio de sesión y cambio de contraseña
Cabeceras de seguridad	Configuración insegura	Helmet con política de seguridad de contenido (CSP) y comprobación de origen
Copias de seguridad	Pérdida de datos	Comando npm run backup que copia la base aunque el servidor esté activo

Pruebas automatizadas. Ejecutamos el comando npm test, que corre tres conjuntos de pruebas sobre bases de datos temporales para no alterar los datos reales. La prueba de humo verificó la autenticación, los permisos, las publicaciones y las suscripciones. La prueba de integración superó 72 comprobaciones HTTP que cubren la configuración inicial, el control de acceso por roles, el flujo editorial, la búsqueda, la moderación, los guardados, la persistencia y la revocación de sesiones. La prueba de operación confirmó que las cuentas se crean y recuperan desde la terminal sin mostrar la contraseña, y que las sesiones y los artículos se conservan después de reiniciar el servidor.

La prueba de navegador, con Playwright, recorrió el flujo editorial completo en Chrome, junto con la búsqueda, los filtros, los guardados, la moderación, el perfil, la advertencia por cambios sin guardar y el cierre de sesión. También comprobó que ninguna página se desborda en anchos de 360, 390, 768, 1024 y 1440 píxeles (ver Figura 5) y pasó auditorías automáticas de accesibilidad con axe-core en la portada, la configuración, el editor, el artículo, la administración y la vista móvil.

Figura 5 Portada de ByteGuard en celular



Nota. Captura de la parte superior de la portada en un ancho de 390 píxeles, generada por la prueba automatizada de navegador.

Limitaciones. El blog funciona en un computador local; publicarlo en internet requiere contratar un dominio, un servicio de alojamiento y un certificado HTTPS. El sitio no envía correos, de modo que la recuperación de contraseña se hace con una herramienta de terminal que solo puede usar quien administra el equipo. La interfaz todavía no tiene un botón para eliminar cuentas. Las auditorías automáticas de accesibilidad no reemplazan una revisión con usuarios reales, y aún no medimos con una encuesta el impacto del blog en los hábitos de la comunidad.

Conclusiones

Cumplimos el objetivo general: ByteGuard funciona como un blog editorial de ciberseguridad en español donde la comunidad de la institución puede leer, escribir y publicar artículos revisados. Los seis artículos iniciales tratan riesgos concretos que enfrentan los estudiantes, como el phishing, las contraseñas débiles y la información que entregan a las herramientas de inteligencia artificial.

El flujo de revisión editorial resultó la decisión de diseño más importante. Abrir el blog a cualquier escritor sin control habría permitido publicar consejos falsos o datos privados; con la revisión, cada artículo pasa por un administrador antes de llegar al lector, y el historial deja constancia de quién hizo qué.

Construir un sitio sobre ciberseguridad nos obligó a aplicar lo que predicamos. Las prácticas del OWASP Top 10 dejaron de ser una lista teórica cuando tuvimos que decidir cómo guardar una contraseña, qué hacer con el HTML que escribe un usuario o cómo cerrar la sesión de una cuenta suspendida.

Las pruebas automatizadas nos ahorraron tiempo y errores. Cada cambio en el código podía romper otra parte del sistema, y las 72 comprobaciones de la API, junto con las pruebas de navegador, nos avisaban antes de que el error llegara al usuario.

Node.js, Express y SQLite resultaron adecuados para un proyecto escolar. El blog se instala con dos comandos, no depende de XAMPP ni de un servidor de base de datos aparte, y cualquier computador con Node.js 20 puede ejecutarlo.

Como trabajo futuro proponemos publicar el blog en un dominio con HTTPS para que la comunidad lo use desde casa, aplicar una encuesta antes y después de su uso para medir el cambio en los hábitos de seguridad de los estudiantes, agregar la recuperación de contraseña por correo y abrir convocatorias para que cada grado escriba artículos.

Referencias

Congreso de Colombia. (1982). *Ley 23 de 1982, sobre derechos de autor*. Diario Oficial No. 35.949. http://www.secretariasenado.gov.co/senado/basedoc/ley_0023_1982.html

Congreso de Colombia. (2009). *Ley 1273 de 2009, por medio de la cual se modifica el Código Penal, se crea un nuevo bien jurídico tutelado denominado "de la protección de la información y de los datos"*. Diario Oficial No. 47.223. http://www.secretariasenado.gov.co/senado/basedoc/ley_1273_2009.html

Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

Congreso de Colombia. (2013). *Ley 1620 de 2013, por la cual se crea el Sistema Nacional de Convivencia Escolar*. Diario Oficial No. 48.733. http://www.secretariasenado.gov.co/senado/basedoc/ley_1620_2013.html

Congreso de Colombia. (2018). *Ley 1915 de 2018, por la cual se modifica la Ley 23 de 1982 y se establecen otras disposiciones en materia de derecho de autor y derechos conexos*. Diario Oficial No. 50.652. http://www.secretariasenado.gov.co/senado/basedoc/ley_1915_2018.html

ESET. (s.f.). *WeLiveSecurity*. Recuperado el 1 de noviembre de 2026, de https://www.welivesecurity.com/es/

Express. (s.f.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. Recuperado el 1 de noviembre de 2026, de https://expressjs.com/

FIDO Alliance. (s.f.). *Passkeys*. Recuperado el 1 de noviembre de 2026, de https://fidoalliance.org/passkeys/

Gruber, J. (2004). *Markdown*. Daring Fireball. https://daringfireball.net/projects/markdown/

Instituto Nacional de Ciberseguridad [INCIBE]. (s.f.). *Oficina de Seguridad del Internauta*. Recuperado el 1 de noviembre de 2026, de https://www.incibe.es/ciudadania

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Ministerio de Tecnologías de la Información y las Comunicaciones [MinTIC]. (s.f.). *En TIC Confío+*. Recuperado el 1 de noviembre de 2026, de https://www.enticconfio.gov.co/

National Institute of Standards and Technology [NIST]. (s.f.). *Cybersecurity*. En *Computer Security Resource Center Glossary*. Recuperado el 1 de noviembre de 2026, de https://csrc.nist.gov/glossary/term/cybersecurity

OpenJS Foundation. (s.f.). *Acerca de Node.js.* Recuperado el 1 de noviembre de 2026, de https://nodejs.org/es/about

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Provos, N. y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference* (pp. 81-91). USENIX Association.

SQLite. (s.f.). *About SQLite*. Recuperado el 1 de noviembre de 2026, de https://www.sqlite.org/about.html

WordPress.org. (s.f.). *Roles and capabilities*. Recuperado el 1 de noviembre de 2026, de https://wordpress.org/documentation/article/roles-and-capabilities/

World Wide Web Consortium [W3C]. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/